

Corrección del Bucle Infinito en el Onboarding (“Perfil de Rockstar”)

Problema reportado

Al completar el formulario de onboarding y pulsar **“FINALIZAR REGISTRO”**, la pantalla se quedaba congelada: el formulario nunca avanzaba y volvía a aparecer una y otra vez (bucle). En las DevTools se veían **decenas de peticiones a Firestore (firestore.googleapis.com) abortadas con NS_BINDING_ABORTED**, repitiéndose continuamente.

Causa raíz

En `src/App.tsx`, la función `handleOnboardingComplete` **esperaba (await) a que la escritura en Firestore terminara ANTES de actualizar el estado local y avanzar la UI**:

```
// ANTES (problemático)
await userService.updateUser(user.id, dataWithFlag); // <- se quedaba colgado
setUser(updatedUser);
setNeedsOnboarding(false);
setActiveTab('home');
```

Firestore está configurado con `experimentalForceLongPolling: true` (`src/services/firebase.ts`) para funcionar en entornos con proxy/sandbox. En redes inestables, la escritura (`setDoc ... { merge: true }`) **no resolvía la promesa** (se reintentaba constantemente vía long-polling, generando el storm de peticiones `NS_BINDING_ABORTED`). Como el `await` nunca terminaba, **el estado local nunca se actualizaba**, `needsOnboarding` seguía en `true` y el formulario permanecía en pantalla → bucle / pantalla congelada.

Solución aplicada

1. `src/App.tsx` — Avance optimista (fix principal)

Se invirtió el orden: **primero** se actualiza el estado local y se avanza la UI, y **después** se persiste en Firestore en segundo plano (sin bloquear):

```
const handleOnboardingComplete = async (data: Partial<User>) => {
  if (!user) return;

  const dataWithFlag = { ...data, onboardingCompleted: true };
  const updatedUser = { ...user, ...dataWithFlag };

  // 1) AVANCE OPTIMISTA: la UI avanza siempre, aunque la red falle/cuelgue.
  setUser(updatedUser);
  setNeedsOnboarding(false);
  setActiveTab('home');
  localStorage.setItem('inspira_user', JSON.stringify(updatedUser));

  // 2) PERSISTENCIA EN SEGUNDO PLANO: no bloquea la pantalla.
  userService.updateUser(user.id, dataWithFlag).catch((e) => {
    console.error("Error al guardar onboarding (se reintentará en segundo plano):", e);
  });
};
```

Por qué elimina el bucle:

- La UI avanza inmediatamente al pulsar “FINALIZAR REGISTRO”, sin depender de que la red responda.
- `onboardingCompleted: true` queda guardado de forma inmediata en `localStorage` como respaldo, y la escritura a Firestore ocurre en segundo plano.
- Al recargar / volver a iniciar sesión, el listener de autenticación (`onAuthStateChanged`) lee el flag `onboardingCompleted === true` (o el `current_rank` ya guardado) y **NO vuelve a mostrar el formulario**.

2. `src/types.ts` — Tipo `gender` ampliado

El formulario permite elegir el género “**Otros**”, pero el tipo `User.gender` solo admitía `'Mujer' | 'Hombre'`, lo que rompía el build de TypeScript (`tsc --noEmit / vite build`). Se amplió:

```
// antes
gender?: 'Mujer' | 'Hombre';
// después
gender?: 'Mujer' | 'Hombre' | 'Otros';
```

Verificación realizada

1. `npm run lint ( tsc --noEmit )` → **OK, sin errores**.
2. `npm run build ( vite build )` → **OK, build exitoso**.
3. Prueba funcional en local (`http://localhost:3000`) con el usuario de prueba:
 - Inicio de sesión con email/contraseña → aparece “Perfil de Rockstar”.
 - Se completó el formulario (Género, Rango, País=México, Estado=Ciudad de México, Ciudad=Benito Juárez, fecha, teléfono) y se pulsó “FINALIZAR REGISTRO”.
 - **El formulario avanzó de inmediato a la pantalla de inicio** (sin bucle ni congelamiento).
 - Tras **recargar la página (F5)**, la app mostró el home con el saludo “¡HOLA, LIDER!” (el `customAddress` guardado) y **el formulario de**

onboarding NO reapareció, confirmando que los datos se persistieron correctamente con `onboardingCompleted: true`.

ACTUALIZACIÓN — Recorrido completo (login → onboarding → logout → re-login)

Al ejecutar el recorrido COMPLETO de prueba (no solo recargar con F5, sino **cerrar sesión y volver a iniciar sesión**) se descubrió que el formulario **SÍ volvía a aparecer** en el segundo login. La investigación reveló la causa raíz real.

CAUSA RAÍZ REAL (infraestructura)

Se probó la conectividad a Firestore directamente desde el entorno (script con el mismo `firebase-applet-config.json`). El login con Firebase Auth **funciona**, pero **la API de Cloud Firestore está DESHABILITADA en el proyecto inspira-bbele**:

```
7 PERMISSION_DENIED: Cloud Firestore API has not been used in project
  inspira-bbele before or it is disabled.
  ... The client will operate in offline mode ...
```

Consecuencias de esto:

- **Ninguna escritura a Firestore persiste** (por eso `onboardingCompleted: true` nunca llegaba a la nube).
- Es el origen del **storm de peticiones** `NS_BINDING_ABORTED` que se veía en DevTools: el cliente intenta conectar al backend de Firestore, falla y reintenta en bucle.
- Al recargar con F5 el formulario NO reaparecía solo porque `localStorage` (`inspira_user`) lo conservaba. Pero **el logout limpia `inspira_user`**, así que al re-loguear el listener `onAuthStateChanged` consultaba Firestore (offline), trataba al usuario como nuevo y **volvía a mostrar el formulario**.

⚠ Acción requerida del lado del usuario (fix definitivo de la nube): Para que los datos se guarden en la nube y persistan entre dispositivos, hay que **habilitar la API de Cloud Firestore** en el proyecto Firebase:
<https://console.developers.google.com/apis/api/firestore.googleapis.com/overview?project=inspira-bbele>
 y crear la base de datos Firestore. Una vez habilitada, el código actual (que ya escribe en Firestore en segundo plano) persistirá correctamente sin más cambios.

FIX DE CÓDIGO (resiliencia ante Firestore caído)

Como el código debe funcionar aun cuando Firestore esté inaccesible, se añadió una **capa de respaldo del perfil POR USUARIO** que **sobrevive al cierre de sesión**:

src/App.tsx

1. **Helpers** `loadLocalProfile` / `saveLocalProfile` (clave `inspira_profile_<uid>`). A diferencia de `inspira_user`, esta clave **NO se borra en el logout**, por lo que el perfil del MISMO usuario se recupera al volver a entrar.
2. **handleOnboardingComplete**: además del avance optimista, ahora guarda el perfil completo con `saveLocalProfile(user.id, dataWithFlag)`.
3. **Listener** `onAuthStateChanged` (las 3 ramas: documento en la nube, sin documento, y error/offline): ahora **fusiona el respaldo local por-uid**. Si localmente ya consta `onboardingCompleted: true` (o `current_rank`), el usuario entra directo al home y **el formulario no reaparece**, aunque Firestore no responda.

Verificación del recorrido COMPLETO (en local)

Probado en `http://localhost:3000` con `alexis.correa026@gmail.com`:

1. **Storage limpiado** → estado de usuario nuevo.
2. **Primer login** → aparece "Perfil de Rockstar" (correcto).
3. **Formulario completado** (Mujer / Lider / Consultora / México / Ciudad de México / Benito Juárez / 01 Ene 1995 / 5512345678) → **FINALIZAR REGISTRO** → **avanzó al home sin bucle**. Se confirmó en `localStorage` la clave `inspira_profile_<uid>` con TODOS los datos y `onboardingCompleted: true`.
4. **Reload (F5)** → sigue en home "¡HOLA, LIDER!", sin formulario.
5. **Logout** → vuelve a la pantalla de login. Se verificó que `inspira_user` e `inspira_auth` quedaron en `null`, pero `inspira_profile_<uid>` **persiste**.
6. **Segundo login (prueba crítica del bug) → fue directo al home; el formulario NO reapareció**. Se verificaron los datos restaurados en `inspira_user`: `onboardingCompleted:true, gender:"Mujer", current_rank:"Consultora", country:"México", state:"Ciudad de México", city:"Benito Juárez", birthDate:"1995-01-01", phone:"5512345678", customAddress:"Lider"`.

Resultado: BUG SOLUCIONADO en el flujo local. La persistencia en la nube (Firestore) quedará activa automáticamente cuando se habilite la API de Cloud Firestore en el proyecto (ver acción requerida arriba).

Archivos modificados

- `src/App.tsx`:
- `handleOnboardingComplete` → avance optimista + `saveLocalProfile`.
- `Helpers` `profileCacheKey` / `loadLocalProfile` / `saveLocalProfile`.
- `Listener` `onAuthStateChanged` → fusión del respaldo local por-uid en sus 3 ramas.
- `src/types.ts` — `gender` ahora incluye `'Otros'`.